MonoSLAM

MonoSLAM is a monocular EKF SLAM in the line of Davison et al. [4]. It uses one camera and no other sensor, and a single extended Kalman filter holds both the camera and the map. A chessboard starts it and fixes the scale, and from then on it tracks the natural features of the scene.

Section 1 describes the frame loop. Sections 2 to 4 cover the state, the motion model and the features. Sections 5 and 6 explain how the features are found in each frame, and section 7 how the update uses them. Section 8 covers the map, and section 9 how it is found again when tracking fails. Section 10 gives the results, and section 11 explains why a filter cannot go much further.

1 The frame loop

MonoSLAM asks the camera for 320×240 frames, which it delivers at about 100 Hz with a fixed exposure of 5 ms, and works on them without resizing (`FrameSource`). All the lengths in pixels in this document refer to these frames, including the ones first measured on 640×480 frames, which have been halved. A quarter of the pixels makes each frame about twice as fast to process, with the same accuracy (section 10).

The camera is calibrated on 640×480 frames. This is the largest frame size the camera gives with the same field of view, and the corners are more precise there. A larger mode with the same aspect ratio does not necessarily have the same field of view: at 1024×768 this camera sees 12% more in each direction. The calibration is then rescaled to the small frames: a pixel center x becomes $(x + \frac{1}{2})s - \frac{1}{2}$, where s is the scale factor, and the distortion, which acts on normalized coordinates, stays the same.

Every frame goes through the same steps, in `main.cpp`:

1. `ekf.predict(dt)` moves the camera with the motion model over the real time between the two frames, taken from their capture times. Live, frames are dropped while one is being processed, so this interval can span several frame periods.

2. `tracker.predict(gray)` uses the optical flow to move the *seed* of every feature, the place where it is expected in the new image (section 5).

3. If the previous frame was weak (section 9), `keyframes.match` searches the features where the most similar keyframes saw them. If that fails, `matchGlobally` searches them anywhere in the frame by their appearance. When either succeeds, the camera is reset to the pose found. Each feature that was found gets its match as its new seed, and every other feature gets its projection from that pose.

4. `ekf.observe()` predicts which features are in view and where. `matchFeatures` searches each of them around its seed, or around its prediction if it is coming back into view (section 6), and `ekf.update` decides with 1-point RANSAC which matches to use (section 7).

5. If the update used at least four features, each seed is replaced by the projection of its feature from the updated state. Otherwise the seeds keep the positions given by the optical flow.

6. Unless the frame is weak, the map is maintained. Features that are rarely matched are removed, and new ones are added where the frame has none (section 8). The frame also becomes a keyframe if it was taken far enough from all the others.

Together, these steps form a loop: images, seeds, search regions, matches, filter, projections, and back to the seeds. Step 5 closes the loop through the filter, which keeps the seeds from drifting. Step 2 keeps the loop running through the optical flow when the filter cannot.

step	reads	writes
`ekf.predict`	state	state
`tracker.predict`	images, seeds	seeds
`Keyframes::match`, `matchGlobally`	image, state, keyframes	state, seeds
`matchFeatures`	image, predicted state, seeds	matches
`ekf.update`	state, matches	state
`tracker.correct`	state	seeds
`Mapper`	image, state	state, seeds
`Keyframes::add`	image, state	keyframes

All the information reaches the filter through its own models and its own update. The seeds only determine where the features are searched. The relocalization of step 3 is the only place where the camera is set without an update.

2 State and conventions

The state is

$$x = \begin{pmatrix} r \\ e \\ v \\ \omega \\ y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^{12+6N}, \tag{1}$$

where $r \in \mathbb{R}^3$ is the camera position in world coordinates, $e \in \mathbb{R}^3$ an orientation error (see below), v the linear velocity in world coordinates, ω the angular velocity in camera coordinates, and $y_i \in \mathbb{R}^6$ the features, in inverse depth (section 4). The world frame is that of the chessboard, which lies on the plane $z = 0$. Lengths are in cm and angles in rad.

The orientation. The orientation is a rotation matrix R, the change of basis from camera to world coordinates. A point with camera coordinates X_c has world coordinates

$$X_w = R\,X_c + r, \tag{2}$$

so the columns of R are the camera axes expressed in the world, and $R_{cw} = R^\top$ goes from world to camera. R is not part of the state vector. The filter keeps it alongside, and the state holds a small rotation e of the camera on top of it, so that the orientation is

$$R\,\exp([e]_\times), \tag{3}$$

where $\exp([e]_\times)$ is the rotation by $\|e\|$ radians about e (Rodrigues' formula, `rotation`) and $[e]_\times$ is the cross-product matrix (`skew`). P holds the uncertainty of e, not of R. After every update, R absorbs e, $R \leftarrow R\exp([e]_\times)$, and e is reset to zero. All the Jacobians are therefore taken at $e = 0$, where $\exp([e]_\times) \approx I + [e]_\times$. This is the error-state, or multiplicative, EKF [7].

When R absorbs e, the covariance of the error must also be expressed about the new reference. An error d about $R\exp([e]_\times)$ is an error $G\,d$ about R, with $G = I - \frac{1}{2}[e]_\times$, so the orientation rows and columns of P are multiplied by G. For the small e of a single update this looks negligible, but it is first order in e. Without it, runs diverge within ten frames from those of a quaternion filter, which applies the same correction implicitly.

Unlike a quaternion, e has exactly as many entries as the orientation has degrees of freedom, so there is no unit norm to maintain and P is never singular. Unlike a rotation vector for the whole orientation, e stays small, far from the angle of π where a rotation vector wraps around and its covariance stops making sense.

Composing rotations. When the camera rotates there are two camera frames, the previous one C and the new one C'. The new orientation $R' = R\exp([w]_\times)$ gives

$$X_w = R'\,X_{c'} + r = R\,\exp([w]_\times)\,X_{c'} + r, \tag{4}$$

a chain of two changes of basis: $\exp([w]_\times)$ maps coordinates in C' to coordinates in C, and R maps those to the world. The rotation leaves its axis w fixed, so w has the same coordinates in C and C', and it is a rotation in camera coordinates. In the world it is $R\,w$, and $R\exp([w]_\times) = \exp([Rw]_\times)R$. The motion model composes rotations on the right, so ω is in camera coordinates, and so is e.

The right Jacobian. To first order, a small change d of the rotation vector of $\exp([\theta]_\times)$ is a small rotation in its own frame,

$$\exp([\theta + d]_\times) \approx \exp([\theta]_\times)\exp([J_r(\theta)\,d]_\times), \qquad J_r(\theta) = I - \frac{1-\cos\varphi}{\varphi^2}[\theta]_\times + \frac{\varphi - \sin\varphi}{\varphi^3}[\theta]_\times^2, \tag{5}$$

with $\varphi = \|\theta\|$ (`rightJacobian`). $J_r(0) = I$, and for the rotation during one frame, $\varphi \approx 0.05$, it differs from I by a few percent.

3 The motion model

Between two frames the velocities change by an unknown amount, the result of a random acceleration that stays constant during the step: $V = a\,\Delta t$ and $\Omega = \alpha\,\Delta t$, with $a \sim \mathcal{N}(0, \sigma_a^2 I_3)$ and $\alpha \sim \mathcal{N}(0, \sigma_\alpha^2 I_3)$. The velocities also decay towards zero, with time constants τ_v and τ_ω. With $e_v = e^{-\Delta t/\tau_v}$ and $h_v = \tau_v(1 - e_v)$, and likewise e_ω and h_ω,

$$\begin{aligned} r_{k+1} &= r_k + h_v\,v_k + \tfrac{1}{2}V\,\Delta t, \\ R_{k+1} &= R_k \exp\left(\left[h_\omega\,\omega_k + \tfrac{1}{2}\Omega\,\Delta t\right]_\times\right), \\ v_{k+1} &= e_v\,v_k + V, \\ \omega_{k+1} &= e_\omega\,\omega_k + \Omega, \\ y_{i,k+1} &= y_{i,k}, \end{aligned} \qquad Q = \Delta t^2 \begin{pmatrix} \sigma_a^2 I_3 & 0 \\ 0 & \sigma_\alpha^2 I_3 \end{pmatrix}, \tag{6}$$

where Q is the covariance of (V, Ω) (`MotionModel`).

The decay. h is the distance a decaying velocity covers in one step, divided by its value at the start of the step. As $\tau \to \infty$, $e \to 1$ and $h \to \Delta t$, which is the constant velocity model of Davison et al. [4]. As $\tau \to 0$ the velocity vanishes immediately, and the camera is predicted to stay where it is. A hand-held camera keeps moving between consecutive frames, but its velocity does not stay the same for long. The decay also limits how far the prediction can drift when nothing is measured.

The factor $\frac{1}{2}$. If the acceleration is constant during the step, the velocity changes linearly from v_k to v_k+V, and the position moves by the mean velocity over the step, $r_{k+1} = r_k+(v_k+\frac{1}{2}V)\,\Delta t$. The model of Davison et al. [4] uses $(v_k + V)\,\Delta t$ instead, an impulse at the start of the step. Both predict the same mean. They differ in how much of the noise reaches the position: a variance of $\frac{1}{4}\sigma_a^2\Delta t^4$ here, against $\sigma_a^2\Delta t^4$. The rotation gets the same factor, which is exact for a fixed axis and correct to first order otherwise.

Linearization. The prediction is linearized at the mean angular velocity, which is not zero. With $\theta = h_\omega\,\omega_k$, the orientation becomes $R_{k+1} = R_k \exp([\theta]_\times)$, and the error after the step is the e_{k+1} that satisfies

$$R_{k+1} \exp([e_{k+1}]_\times) = R_k \exp([e_k]_\times) \exp([\theta + h_\omega\, d\omega]_\times), \tag{7}$$

for an error $d\omega$ of the angular velocity. Using the right Jacobian, and moving $\exp([e_k]_\times)$ across $\exp([\theta]_\times)$ as above, gives $e_{k+1} \approx \exp(-[\theta]_\times)\,e_k + h_\omega J_r(\theta)\,d\omega$. The Jacobian F of the camera part is therefore the identity except for

$$\frac{\partial r_{k+1}}{\partial v_k} = h_v I_3, \quad \frac{\partial e_{k+1}}{\partial e_k} = \exp([\theta]_\times)^\top, \quad \frac{\partial e_{k+1}}{\partial \omega_k} = h_\omega J_r(\theta), \quad \frac{\partial v_{k+1}}{\partial v_k} = e_v I_3, \quad \frac{\partial \omega_{k+1}}{\partial \omega_k} = e_\omega I_3. \tag{8}$$

The noise enters through the velocities, so $W = \partial f/\partial(V, \Omega)$ is 12×6, with $\frac{1}{2}\Delta t\, I_3$ and I_3 in the rows of r and v, and $\frac{1}{2}\Delta t\, J_r(\theta)$ and I_3 in those of e and ω. Only the camera moves, so P becomes

$$\begin{pmatrix} FP_{cc}F^\top + WQW^\top & FP_{cf} \\ P_{fc}F^\top & P_{ff} \end{pmatrix}, \tag{9}$$

where c stands for the camera and f for the features. P is then symmetrized, so that rounding errors do not accumulate over thousands of steps. The tests *Process Jacobian* and *Process noise Jacobian* compare F and the orientation block of W with finite differences of the prediction, with the error measured from the nominal prediction.

Values. $\sigma_a = 1000$ cm/s^2, $\sigma_\alpha = 15$ rad/s^2, $\tau_v = 0.3$ s and $\tau_\omega = 0.2$ s. In one frame the velocities can change by about 33 cm/s and 0.5 rad/s, and without measurements they halve in about 0.2 and 0.14 s. The time between frames comes from their capture times, or is 1/30 s for recordings that have none.

Why this model. The chessboard ground truth (section 10) gives the camera pose in every frame where the whole board is detected. Running a Kalman filter on these poses with each candidate model, with the noise fitted by maximum likelihood over three sequences, measures how well the model predicts the next frame (negative log-likelihood, lower is better):

model	position	orientation
no velocity, a random displacement	1827	−10565
impulse [4]	674	−11655
constant acceleration during the step	649	−11710
same, with decay, $\tau = 10$ s	639	−11718
same, with decay, $\tau = 0.3$ s	595	−11858

Having velocities in the state matters most, by more than a thousand: the motion of the camera persists from one frame to the next. How the noise enters them matters little, and a decay of about 0.3 s fits best, at the cost of one more parameter. The motion itself only needs about 90 cm/s^2 and 5.5 rad/s^2, but the filter needs larger values, for two reasons. First, the noise also has to absorb the errors of the filter. Second, the fastest frames are the most blurred, so the board is not detected in them and this fit never sees them.

4 Inverse depth features

A feature uses the inverse depth parametrization of Civera et al. [2]:

$$y = (a, \theta, \phi, \rho) \in \mathbb{R}^6, \qquad p = a + \frac{1}{\rho}\, m(\theta, \phi), \qquad m(\theta, \phi) = \begin{pmatrix} \cos\phi \sin\theta \\ -\sin\phi \\ \cos\phi \cos\theta \end{pmatrix}, \tag{10}$$

where the anchor a is the position of the camera when the feature was first seen, θ and ϕ are the azimuth and elevation of the ray from a to the point, in world coordinates, and ρ is the inverse of the distance along that ray.

Why. A point seen once is only a ray: its depth stays unknown until the camera has moved enough. Along the ray the uncertainty of the depth is very asymmetric, since a point at 50 cm may be at 25 cm or at infinity. A Gaussian on p cannot describe that, but a Gaussian on ρ can: $\rho \in [0, 0.04]$ covers everything from 25 cm to infinity. The projection is also much closer to linear in ρ than in the depth. A feature can therefore enter the filter in the frame where it is first seen, with a large σ_ρ, and constrain the orientation of the camera long before its depth is known.

Observation. The vector

$$h_c = R^\top \big(\rho\,(a - r) + m(\theta, \phi)\big) \tag{11}$$

is the point in camera coordinates, scaled by ρ. The scale does not change its pixel, and it keeps h_c well defined at $\rho = 0$, for points at infinity. The pixel is the projection of h_c through the lens distortion of the camera, and its Jacobians are those of h_c with respect to r, e and y, multiplied by the Jacobian of the projection. Rotating the camera by e turns h_c into $\exp(-[e]_\times) h_c \approx h_c + [h_c]_\times e$, so $\partial h_c / \partial e = [h_c]_\times$ (`inverse_depth::toCamera`, `CameraModel::projectCameraPoint`). A feature is in view if h_c is in front of the camera and its pixel falls inside the image.

No conversion to points. Civera et al. [2] convert a feature to a point $p \in \mathbb{R}^3$ once its depth is known well enough, which saves three entries of the state. Here features are never converted, and keep their six parameters until they are removed. The observation model, the map and the code stay uniform, at the price of a slightly larger state. `EKF::isWellLocated` identifies the features whose depth is known, $\rho > 0$ and $\sigma_\rho < 0.1\,\rho$, which is where Civera et al. would convert them. `inverse_depth::toPoint` gives the point when one is needed, for drawing or for warping patches. A feature with $\rho \le 10^{-3}$ is placed 10 m along its ray.

Initialization. The first camera pose comes from the whole chessboard, with PnP, and is accepted if it explains all the corners within 3 px and with an RMS below 1 px. Five of the corners become the first features: the four outermost ones and the one closest to the center. They are placed at their known positions p_i, with the anchor at the initial camera position, $y_i = (r_0, \theta_i, \phi_i, \rho_i)$ (`fromPoint`). The covariance of $(\theta_i, \phi_i, \rho_i)$ is $J\,(0.05\, I_3)\, J^\top$, where J is the

Jacobian from `fromPointJacobian`, which amounts to a variance of 0.05 cm^2 on each axis of the point. The anchor has no uncertainty, since it is a constant and not a measurement. The camera starts with standard deviations of 0.22 cm, 0.02 rad, 30 cm/s and 1.5 rad/s, and no velocity. This is where the scale of the map comes from. The chessboard plays no further role.

New features. A feature first seen at pixel u from the pose (r, R), with an inverse depth ρ_0, is

$$y = \big(r,\ \theta(d),\ \phi(d),\ \rho_0\big), \qquad d = R\,\pi^{-1}(u), \tag{12}$$

where $\pi^{-1}(u)$ is the ray of the pixel in camera coordinates, found by undistorting it. Rotating the camera by e turns d into $R\exp([e]_\times)\pi^{-1}(u) \approx d - R\,[\pi^{-1}(u)]_\times e$. With $J = \partial y/\partial(r, e, u, \rho_0)$, of size 6×9 (`inverse_depth::fromPixel`), the state grows to

$$x \leftarrow \begin{pmatrix} x \\ y \end{pmatrix}, \qquad P \leftarrow \begin{pmatrix} P & P_{:,c}\,J_c^\top \\ J_c\,P_{c,:} & J_c\,P_{cc}\,J_c^\top + J_u\,\sigma_u^2\,J_u^\top + J_\rho\,\sigma_\rho^2\,J_\rho^\top \end{pmatrix}, \tag{13}$$

where c stands for the six entries of the camera position and orientation error, σ_u is the pixel noise, and σ_ρ the uncertainty of the inverse depth (`EKF::addFeature`). Because the new feature is correlated with the pose it was added from, a later update that corrects that pose also corrects the feature.

5 The seeds

Every feature has to be found again in each new image, and the first question is where to look. In EKF SLAM the search region is the 3σ ellipse of the innovation covariance around the prediction, the active search of Davison et al. [4]. That only works if the ellipse contains the feature and nothing that looks like it, and a hand-held camera jerks. On the first sequence the chessboard corners move up to 19 px between frames, and a constant velocity prediction only brings the median error down from 5 to 3 px, with 11 px at the 99th percentile. With the active search, the first two sequences are lost in the frames where the motion changes by 10 px from one frame to the next. The motion model cannot predict such changes, but the images can.

`FlowTracker` follows the image motion of every feature, its *seed*, without any knowledge of the filter. It keeps the previous image and one seed per feature, including those out of view, and in every frame (`predict`) it does three things:

1. *It tracks generic corners.* The corners of the previous image are tracked into the new image and back with pyramidal Lucas–Kanade [8], with six levels and an 11×11 window. A corner is kept if it comes back within 0.5 px of where it started, which removes most corners lost to blur or occlusion. The corners carry over from frame to frame, and new ones are only detected when fewer than 120 are left (`goodFeaturesToTrack`, quality 0.01, at least 4 px from each other and from the remaining ones), up to 150. Detecting them again in every frame made the flow 40% slower. The six levels matter: with four, the second sequence is lost, because the flow cannot follow jumps of more than 50 px.

2. *It moves each seed with its neighbors.* An affine map is fitted with RANSAC to the motion of the 24 tracked corners closest to the seed (inliers within 1.5 px), and the seed is moved by that map. With fewer than 6 corners, or if the fit fails, the seed stays where it is. Seeds outside the image are moved in the same way, by the corners closest to them.

3. *It lets the seed's own flow take over.* The seeds in view are also tracked themselves, with a 15×15 window. If the result lands within 3 px of the affine position, it replaces it, since the flow of a single point is more precise. If not, the seed has probably slipped to a similar-looking neighbor, and the affine position is kept.

An affine map is the first-order approximation of the image motion around a point. Using it assumes that the seed moves like the corners around it, which is true for a locally smooth surface, and for any scene while the camera mostly rotates. At a depth discontinuity, with the camera translating, the fit follows whichever surface has more corners. The test *Flow follows depth layers* checks that two regions moving apart each carry their own seeds.

After the update, if at least four features were used, `correct` replaces each seed with the projection of its feature from the updated state. A seed that slipped to a neighbor is corrected this way, since the other features fix the pose. With fewer features the pose is not well constrained, and the seeds keep the positions given by the optical flow.

6 The patch search

Each feature keeps the 11×11 px patch around it in the frame where it was first seen. To find it in a new frame (`findPatch`), the patch is first warped to how it would look from the predicted pose. The patch is treated as a piece of the plane through the feature with its normal: the chessboard's for the five chessboard corners, and facing the first camera for all other features. The corners of the patch are mapped through the whole camera model, distortion included, and the warp is the homography that maps them.

The warped patch is searched within ± 8 px of the seed, or of the prediction if the seed is out of view, by zero-mean normalized cross-correlation over the mask of the warped patch. The best peak is kept if it scores at least 0.6 and no other peak, meaning a local maximum more than 1 px away, scores within 0.05 of it. Two nearly equal peaks mean a repeated texture or an edge, and which one wins then depends on tiny differences in the prediction. Every feature found is returned, and the update decides which ones to use.

Subpixel alignment. A parabola through the peak and its neighbors gives a first subpixel position, but it is biased towards whole pixels. The warped patch is then aligned with the frame by the inverse compositional Lucas–Kanade algorithm [1] for a translation, with at most 10 iterations (`alignPatch`). At every step the frame is brought to the mean and contrast of the patch, as the correlation does. The alignment is abandoned if it wanders more than 1 px away from the peak. It succeeds for 84 to 99% of the matches and moves them by a median of 0.12 to 0.18 px. The fit also tells how well the patch was aligned. Its covariance is the variance of the residuals over the Hessian of the patch gradients,

$$\Sigma_a = \frac{\sum_k e_k^2}{n-2} \Big(\sum_k \nabla T_k \nabla T_k^\top \Big)^{-1}, \tag{14}$$

which is large when the patch does not look like the frame, or when it lacks texture in some direction, as along an edge. For the median match, its larger standard deviation is 0.1 px. On its own, the alignment does not change the results beyond the spread of the runs: the precision of the matches is not what limits them. Its covariance does improve them (section 7).

Returning features. A feature that was out of view for at least 10 frames is *returning* until the update uses it again. Its seed is useless, because the flow could not follow the feature while it was away, so it is searched around its prediction instead. The region is 3σ of the larger standard deviation of its innovation, as in the active search of Davison et al. [4], but no smaller than ± 8 px and no larger than ± 20 px. While the camera is away, the uncertainty of the feature grows, and so does its search region, so the feature is found even if the map has drifted. The upper limit keeps a large uncertainty from letting look-alikes into the region: without it, at ± 30 px, some

runs of sequence 6 end more than 3 m off. Over 15 runs, searching returning features this way recovers 17 of the 30 kidnaps, against 9 when they were searched around their seeds.

7 The update

`EKF::observe` predicts the pixels h of the features in view, two rows per feature. It also computes their Jacobian H, the innovation covariance $S = HPH^\top + \sigma^2 I$ with $\sigma = 1.5$ px, and the columns of P that H acts on. H only has the columns of the camera and of the features in view. A pixel depends only on the camera and on its own feature, so most of H is zero, and the product $PH^\top$ skips those zeros.

The noise of each match. σ covers many things the alignment does not see, such as the error of the warp, and it remains the noise of a well aligned match. A match aligned worse gets more noise, in the directions where it was worse (`matchNoise`). With $\Sigma_a = V \operatorname{diag}(\lambda_1, \lambda_2) V^\top$,

$$R = \sigma^2 V \operatorname{diag}\big(\max(1, \lambda_1/\lambda_0), \max(1, \lambda_2/\lambda_0)\big) V^\top, \qquad \lambda_0 = 0.04 \text{ px}^2, \tag{15}$$

which replaces the block $\sigma^2 I$ of the match in S. About one match in ten has an alignment standard deviation above $\sqrt{\lambda_0} = 0.2$ px. No match is trusted more than before, so this can never make the filter more confident. Over 15 runs the kidnaps recovered go from 18 to 22 of 30, the returns of sequence 5 are as good or better, and everything else stays within the spread of the runs. With $\lambda_0 = 0.01$ px^2 the kidnaps recovered were 23, but the second visit of sequence 6 was worse. With 0.0025 px^2, where most matches get extra noise, they were 15, and sequence 3 was lost once.

One EKF step. `correct` applies the standard EKF update with a subset of the rows of the observation,

$$x = x_0 + K\big(z - h(x_0)\big), \qquad K = PH^\top S^{-1}, \qquad P \leftarrow P - KHP, \tag{16}$$

solving $SK^\top = (PH^\top)^\top$ for $K^\top$ by Cholesky. Nothing is updated if S is not positive definite. R then absorbs the orientation error (section 2).

1-point RANSAC. Some of the matches of a frame are wrong, and `EKF::update` decides which ones to use, as in Civera et al. [3]:

1. *Hypotheses.* For every match j, a copy of the state is updated with that match alone,

$$x^{(j)} = x_0 + K_j\big(z_j - h_j(x_0)\big), \qquad K_j = PH_j^\top S_{jj}^{-1}. \tag{17}$$

   A single pixel only fixes two degrees of freedom of the pose. The others move according to their correlations with those two in P, which is the most likely way given the prediction, and that is enough to tell which other matches agree. The hypotheses are states with $e \neq 0$, projected with $R \exp([e]_\times)$.

2. *Support.* The prediction $h(x^{(j)})$ is computed with the full nonlinear model, and every match within 3 px of it supports the hypothesis. The features in view at $x^{(j)}$ need not be the same as those in view at x_0. Rows are therefore paired by the feature they belong to, not by their position, and a match whose feature leaves the image does not support the hypothesis. The hypothesis with the largest support wins, and a tie is broken by the smallest sum of distances.

3. *Low innovation inliers.* The state is updated with the winning support.

4. *High innovation inliers.* A match can be right and still disagree with every single-match hypothesis, if the prediction was far off. After the first update the prediction is much better, so h, H and S are computed again. Every remaining match with a normalized innovation squared below 9.21, the 99% quantile of χ^2_2, is rescued and used in a second update. Updating in two steps is equivalent to updating with both sets at once, up to the linearization.

The thresholds, 3 px and 9.21, are part of the measurement model, together with σ (`MeasurementModel`). Since everything goes through h, H, P and σ, nothing in the update knows about poses or images. Each hypothesis costs one evaluation of the observation model, which is cheap for a dozen features. With hundreds, a random subset of hypotheses would do, as in Civera et al. [3].

The plain update is enough. The projection is not linear, and an iterated EKF, which is Gauss–Newton on the same cost, would follow its curvature. Without a line search it oscillates on this cost. With the few rescued matches of the second stage, which do not determine a pose on their own, it wanders along the directions they leave free. And the curvature it would correct is small for the innovations that 1-point RANSAC lets through.

8 The map

`Mapper` maintains the map at the end of every frame that is not weak.

Adding. While fewer than 12 features are in view, up to two new ones are added per frame:

1. *Where.* A box a quarter of the frame in each dimension slides over it in steps of half its size, staying 10 px from the border. The positions with no feature in view inside are tried in order, the farthest from the existing features first, so that the features keep a roughly constant density over the frame. Davison et al. [4] place the box at random instead, but a fixed order keeps the runs repeatable. A box can be empty because there is nothing to track in it, and then the next one is tried.

2. *Salient.* `goodFeaturesToTrack` proposes up to 10 corners in the box (Shi–Tomasi [10], quality 0.01, at least 5 px apart), 15 px away from the features in view, strongest first.

3. *Textured.* The quality of `goodFeaturesToTrack` is relative to the best corner in the box, so on a blank wall the least blank point still passes. A candidate also needs an absolute amount of texture: the root mean square of the gradient of its patch along its least textured direction. This is the smaller eigenvalue of the gradient matrix of the patch, as in Shi and Tomasi, and it must be at least 4 gray levels per pixel. On the six sequences, the features below 2, 31% of them, were kept for a median of 11 frames and were used in 21% of their searches, while those above 4 were used in about 70%. The correlation used in the next test cannot detect a blank patch: it divides by the contrast of the patch, and what remains is camera noise.

4. *Distinctive.* A candidate is rejected if its patch correlates with some other place within ± 20 px, outside ± 2 px of itself, with a score of 0.8 or more. This keeps new features off the inner corners of the chessboard, and off repeated texture in general, which the search could not tell apart within its region.

5. *Initial depth.* ρ_0 is the inverse of the median distance of the well located features in view, or 0.02 cm^{-1} (50 cm) if there are none. Its uncertainty is $\sigma_\rho = \rho_0$, so the 2σ interval covers everything from a third of that distance to infinity.

6. The feature gets its patch, a descriptor (section 9) and a seed.

Removing. In every frame, each feature that is searched counts an attempt, and a match if the update used it. After 10 attempts, a feature used in less than half of its last 20 is removed from the state, the covariance (`EKF::removeFeatures`), the list of features and the tracker. Only recent attempts count, so a feature that is no longer matched is removed within a third of a second, even if it had been reliable for a long time. This removes occluded features, features on the edges of objects, whose patch changes with the point of view, and features on things that move. A feature that comes back into view after at least 10 frames away starts counting again from zero. Its old failures are no longer relevant, and a single bad frame on its return would otherwise remove it.

Two kinds of features are never removed: the chessboard corners, which hold the scale of the map, and features used in at least 30 frames. A feature used that often is part of the map. When the camera returns after a long excursion, these features are what closes the loop. Their first searches after the return may fail, because the state has drifted, and under the rule above they would be gone before they could pull the state back. Without this exception, sequence 6 comes back to the board after a minute with nothing left of the first visit except the board itself, whose identical corners then match a whole square away.

9 Finding the map again

The search only looks near the seeds and the predictions. When the camera gets somewhere else without the images or the filter following it, as in a kidnap, simulated by cutting frames out of a sequence, the features are no longer where the search looks for them. The features are then searched where the keyframes that resemble the frame saw them, as in Klein and Murray [6] (`Keyframes::match`). If that fails, they are searched by their appearance alone, anywhere in the frame, as in Williams et al. [12] (`matchGlobally`).

Keyframes. After every frame that is not weak and used at least 8 features, the frame becomes a keyframe if no existing keyframe is within 5 cm and 8° of its pose. It is stored shrunk to 80×60 pixels, blurred with $\sigma = 2.5$ of those pixels and with its mean removed, together with the pose (`Keyframes::add`). After a weak frame, the frame is shrunk in the same way, and the three keyframes with the smallest mean squared difference to it are tried in turn. Since a small turn of the camera shifts the whole image, each keyframe is first aligned with the frame by the shift that makes them most similar, within 6 thumbnail pixels. The well located features, except those of the chessboard, are projected with the pose of the keyframe and shifted by the same amount. Each one is searched within 10 px, with its patch warped to that pose. A pose is fitted to the features found, and verified as described below for the descriptors.

The global match needs a detected corner to fall on the feature, and an ORB descriptor that stays similar over the few pixels between the two. On these sequences only 3 or 4 of its 10 to 14 matches are right, even while tracking is good, out of about 9 features in view. The keyframes need neither, because they search with the same patches that track the features, around where those should be. They do need the camera to be back near a pose it had before. On sequence 5, right after the second cut, the keyframe that looks most like the frame is 5 cm and 4° away from it. At the third visit of sequence 6, the camera is 9 to 14 cm from every pose it had, and neither

method finds the map. Over 15 runs, the kidnaps recovered went from 19 to 23 of 30, and the return to the board at frame 836 of sequence 6 is within 10 cm in 13 runs instead of 9.

Descriptors. Every feature gets an ORB descriptor [9] when it is created (`describe`). The descriptor is made of 256 binary comparisons of pixel pairs in the 31×31 patch around the feature. The pairs are rotated by the orientation of the patch, the direction to the centroid of its intensities, which turns with the image. ORB only computes this orientation for the corners it detects itself, so it is computed here. Seen from the side, a feature can be 60 bits away from how it looked when it was created. So while a feature is tracked, its descriptor in the current frame is also kept if it is more than 30 bits away from all those it already has, up to eight (`Mapper::recordAppearance`).

Matching. Up to 1500 corners of the frame are detected with `goodFeaturesToTrack`, as the features were, since ORB's own corners rarely fall on them, and described in the same way. Each descriptor of each well located feature is matched to its nearest corner. The match is kept if that corner is within 64 bits, and clearly closer than the best corner more than 5 px away from it, by a ratio of 0.8. Corners around the same structure look alike, but that is not an ambiguity. The five chessboard corners are left out, since they all look the same. A pose is fitted to the matches with P3P and RANSAC, with 1000 iterations and inliers within 2 px, and at least six matches must agree.

Verification. A wrong pose can agree with six descriptors by chance, so the pose is verified with the patches. The features are projected with it and searched, warped to it, within 4 px of their projections, and at least six must be found within 1.5 px.

When it is used. A frame is *weak* when the update used fewer than four features, or fewer than half of those searched. The state may then be wrong, even if some matches still agree with it: after a kidnap, 1-point RANSAC can keep finding a few look-alikes that agree with a wrong pose.

- After a weak frame, the keyframes or the global match decide. If either finds a pose, `resetCamera` resets the camera to it, with no velocity, a large uncertainty (2 cm, 0.2 rad, 30 cm/s, 1.5 rad/s) and no correlation with the features. The seeds are then replaced by the projections of their features.
- While fewer than half of the features searched are used, the map is left alone. No feature is added, since it would be placed from a wrong pose, and no failed match is counted, since the state, not the features, is to blame for the failures. A frame with fewer than four features used, but most of those searched, is different: the camera is entering a part of the scene that has no features yet, and it needs new ones. Freezing the map there would keep tracking weak and the map frozen indefinitely.

10 Results

Ground truth. The sequences start on the chessboard, and some come back to it. `analyze_pattern` finds the whole board in every frame where it can, and fits the camera pose to its 54 corners. A run is scored in two ways. The first is in pixels: the distance from the detected corners to the projections of the five chessboard features from the estimated pose (`evaluate_tracking.py`). The second is the error of the camera position when the board

comes back into view. The detector misses the blurriest frames, which are the fastest ones, so the annotated frames are checked as well.

The runs are chaotic. A tiny difference, such as changing the pixel noise by one part in a billion, can flip a borderline decision, such as whether a match is accepted or a feature added. From then on, the two runs diverge. A single run cannot tell a better system from a lucky one, so every number below pools 15 runs: five sets of initial chessboard corners, each with three negligible changes of the pixel noise.

sequence	result over 15 runs
1	median 0 frames over 5 px, at most 3
2	median 6 frames over 5 px, at most 8
3	median 2 frames over 5 px, never lost
5, back on the board at frames 336, 514, 566	median errors 1.5, 1.7, 1.9 cm
5, kidnaps: frames 141–249 and 421–479 cut	recovered in 9 and 13 runs
6, board visits at frames 359, 411	median errors 3.7, 1.4 cm
6, board visits at frames 771, 836	median errors 4.9, 1.1 cm
6, board visits at frames 1413, 1549	lost in every run

Sequences 1 to 3 look at the board most of the time. Sequences 4 and 5 leave it after the initialization and track natural features, such as a table, a bowl, a cable, chairs and pictures, with 5 to 20 features per frame. Sequence 5 comes back to the board twice. Sequence 6 is a minute long and comes back three times. The map holds the first time. The second excursion drifts, often by tens of cm by the time the board comes back into view at frame 771. The features kept from the first visit then close the loop: by frame 836, 11 runs out of 15 are within 10 cm. Before the third visit the camera sweeps a blurred, almost textureless sofa, and tracking is lost in every run. The global match does not find the map again before the end. The losses of sequence 3 all come from the same set of initial corners, where the camera swings fast and the board is blurred.

Speed. On sequence 6, while tracking, a frame takes 6 ms, the median of the averages over every second, against 14 ms with the same code on 640×480 frames. At that size the filter took 1 to 3 ms, and most of the time went to the images. The optical flow took 6 to 10 ms, of which 4 ms went to the Lucas–Kanade tracking of the 150 corners forwards and backwards, and up to 3 ms to the affine motion fitted around each seed. The search of the patches took 2 to 6 ms. With a quarter of the pixels, and windows and patches half as wide, the images cost a fraction of that. Over 15 runs, the accuracy is the same within the spread of the runs: the first return of sequence 5 is 1 cm worse, the second visit of sequence 6 is better, and the kidnaps recovered go from 20 to 18 of 30. The subpixel alignment of the matches adds about 1 ms. Three more things keep the time down:

- The masked correlation of the patches is computed from three plain correlations (`maskedCorrelation`). It gives the same score as OpenCV's masked `matchTemplate`, 2.7 times faster.
- The pyramid of each frame for the flow of the corners is built once, and used forwards, backwards, and again as the previous frame. It is built with the window of the corners. With the larger window of the features it would have one level less, and the corners would lose track of fast motions, as they did in 11 of the 15 runs of sequence 3.

- When new features are added, each candidate corner is checked only once. The boxes overlap and are searched again for every feature added, and checking every candidate each time took up to 40 ms in one frame out of 14.

After a weak frame, the keyframes and the global match run, so while the camera is lost a frame takes longer, up to 30 ms on average over a second. Live, a frame that takes longer than the frame period causes the next one to be dropped, which doubles the interval of the prediction, and with it the uncertainty of the predictions. With `--timing` the decoding of the camera frames is reported too, about 1 ms. Each frame that is read is a new image that nothing writes to afterwards, so tracking, the features and the window share it without copying it. The window is drawn and shown by the main thread, while tracking runs in another thread, which only hands over its latest frame (`Display`). At 640×480 the display took 3 ms per frame, and much more with the window maximized, which was enough to drop frames. 150 corners track as well as 300, but on 640×480 frames a 15×15 window with at most 10 iterations lost sequences 2 and 3 in every run.

Limitations.

- At a depth discontinuity, the seed of a feature follows the surface with more corners. Keeping both predictions, the flow of the feature itself and the motion of its neighbors, and letting the update choose, would fix it.
- 8 of the 30 kidnaps are not recovered in time, mostly the first one, where the camera sees, after the cut, a part of the scene it had not seen before. Which kidnaps are recovered changes a lot from run to run: the floating point rounding of the faster correlation alone moved the first kidnap from 13 recoveries to 9, and the second from 10 to 11, out of 15.
- Over long excursions the map drifts, and the filter does not know it: the uncertainty it reports grows far less than the error. This is the limit of the filter itself (section 11).
- The camera has a rolling shutter. On the chessboard frames, the reprojection error is smallest if the rows are exposed over about 10 ms, from top to bottom. Predicting each feature at the time of its row did not help over 15 runs, since the motion in these sequences is slow, and it only moves the corners by about 0.01 px on average. It may matter for fast live motion. `evaluate_tracking.py` would then need it as well, since it projects with a single pose per frame.
- The features used for a long time are never removed, so the map, and the cost of the filter, grow with the explored scene.
- The global match only describes detected corners, which lie a few pixels away from where the descriptor of a feature would match best, and ORB descriptors change a lot over a few pixels. On the return to the board at the end of sequence 6, most visible features match their descriptors well within 4 pixels of their true position, but only one or two at the corners that are described.

11 Where the filter ends

When the camera turns away from the mapped part of the scene, every feature in view is new, and the filter becomes confident about a pose that is wrong. This section shows where that comes from, that none of the remedies from the filtering literature removes it, and that the images themselves contain the information the filter loses.

A simulation. In a simulation, the models of the filter can be made exactly true. The camera follows the motion model of section 3, with its noise drawn as the filter assumes, among 20 000 points on a shell 1.5 to 6 m around the start. The pixels are the true projections plus Gaussian noise with the σ of the filter, data association is perfect, and features are added and dropped as the mapper does, with the five chessboard corners known. The table gives the normalized estimation error squared of the camera pose (NEES, which is 6 for a consistent filter), at 100 Hz, over 20 runs of 10 s:

new features enter with	NEES, last 2 s	position error
no measurements, prediction only	5.1–5.5	
their true inverse depth, $\sigma_\rho \approx 0$	5–6	1.1 cm
an inverse depth drawn with $\sigma_\rho = \rho$, as in MonoSLAM	721	13.7 cm
the same, Jacobians at the true features	5–7	1.4 cm
an inverse depth drawn with $\sigma_\rho = 0.1\,\rho$	5–6	1.6 cm

With exact models and exact matches, the filter is consistent if the features enter with a known depth. If they enter as MonoSLAM adds them, it is over a hundred times too confident, and reports a position uncertainty of 1.4 cm for an error of 13.7 cm. Linearizing the observations of the features at their true parameters, with the camera still at its estimate, makes it consistent again. The cause is therefore the linearization of the young features at a wrong inverse depth. It requires the camera to rotate and translate at the same time, and it happens with noiseless pixels as well. Every update with a young feature gives the camera information computed with a wrong Jacobian, and later updates cannot take it back.

The remedies of filtering. None of these brings the NEES down:

- *The second-order term* of the projection in ρ and in the camera position, added to S, lowers it from 880 to 620 at 30 Hz, and the position error stays the same.
- *A partial (Schmidt) update*, in which the features with $\sigma_\rho > 0.3\,\rho$ only update themselves, with the Joseph form for the covariance, makes it worse. The update of the young feature itself still uses the Jacobian at its wrong ρ.
- *First-estimate Jacobians* for the features make it worse, because the first estimate is the uncertain one.
- *An iterated EKF*, relinearizing about each new estimate, is as consistent as the plain update when the depths are exact, and worse when they are not: 1460 with 3 iterations, and 2874 with 10. A single frame cannot observe ρ, so iterating only commits the camera harder to the wrong value. The linearizations that are wrong belong to the past frames.
- *Delayed initialization* tracks the features outside the filter with Lucas–Kanade, fits their inverse depth along the epipolar line, and lets them in once $\sigma_\rho \leq 0.15\,\rho$, or after 30 frames. On the real sequences it lost to the current filter in every variant. When the camera enters a new part of the scene, the filter is left with 2 to 5 features instead of 8 to 18, and it drifts while it waits.

The invariant EKF fixes the inconsistency that comes from the unobservable directions of SLAM, the global pose and the scale. Here the five chessboard features, which are never removed and have a tiny covariance, already pin those down, and the Jacobians of an invariant EKF would still depend on the depth of the young features.

Two sequences at 100 **Hz.** Sequences 7 and 8 last 18 s each. They start on the board, explore the room and come back. The board is detected less reliably near the border of the frame, where its corners are fitted with an RMS of 2 to 4 px instead of less than 1, so only frames fitted within 1.5 px are scored. In the runs that close the loop, the position error is 1 to 4 cm, about 2 to 3 times the standard deviation the filter reports. On sequence 7, two of the five runs, which differ only in their initial corners, end 80 to 120 cm off.

These two runs go wrong between frames 860 and 1000, when the board leaves the view and the camera pans from the table to a bookshelf. By then all the features in view are new, and the five runs do not agree on the motion:

run	displacement $860 \to 1000$, cm	bundle adjustment, cm
default corners	$(-6.6, -2.1, 18.3)$	$(-4.4, -4.2, 11.3)$
corners a	$(-3.6, -2.4, 16.9)$	
corners c	$(-5.1, -0.6, 15.6)$	
corners b	$(6.1, -3.4, 10.7)$	$(-1.0, -4.4, 9.6)$
corners d	$(3.2, -2.8, 10.0)$	

Runs b and d drift smoothly, by 1 cm every 5 frames. They use 10 to 12 features, with no outlier that matters, and report a position uncertainty of about 1 cm. This is the mechanism of the simulation. Everything that follows is a consequence of it. The features added during the drift do not agree with those from the first part of the map. When the camera later faces a door, old features are predicted in view behind the wall, and most of the searched features fail. The map then freezes (section 9), and tracking runs out of features. The relocalizations then jump between the two parts of the map, by about 45 cm from one frame to the next.

The information is in the images. A bundle adjustment relinearizes every observation at the final estimate. Here it is run over the 140 poses and the new features, with the matches each run used, and with the pose at frame 860 and the older features fixed. Fitted to the matches of run b alone, and to those of the default run alone, which are two independent sets, the camera ends up 3.7 cm apart at frame 1000, where the two filters were 14 cm apart. The median reprojection errors are 0.5 and 0.2 px. The motion is in the images, and the filter loses it, because it cannot revisit an update once it has been made.

Conclusion. This is the result of Strasdat et al. [11], which ended filtering as the method of choice for visual SLAM. For the same computation, keyframes and bundle adjustment beat a filter, because they can relinearize, and because they can afford many more points. Within a filter, the remaining option is to add sensors: a gyroscope would measure the rotation, and the young features could no longer confuse it with the translation. With a single camera, the next step is not a better filter, but tracking against a map maintained by bundle adjustment, as in PTAM [5] and its successors.

12 Correspondence with the code

frame loop, parameters	`main.cpp`
R, exp, $[\cdot]_\times$	`EKF::orientation`, `rotation`, `skew` in `camera.h`
f, F, W, Q, J_r	`MotionModel` in `ekf.h`, `rightJacobian`
σ, update thresholds	`MeasurementModel` in `ekf.h`
prediction	`EKF::predict`
h, H, S	`EKF::observe`
update, 1-point RANSAC	`EKF::update`, `EKF::correct`
state augmentation, removal	`EKF::addFeature`, `EKF::removeFeatures`
relocalization of the camera	`EKF::resetCamera`
inverse depth	`inverse_depth::toCamera`, `fromPixel`, `fromPoint`, `toPoint`
projection	`CameraModel::projectCameraPoint`
initialization	`initializeFromPattern`
patch, descriptors of a feature	`Feature` in `feature.h`
seeds	`FlowTracker` in `tracking.h`
patch search, returning features	`findPatch`, `matchFeatures`
subpixel alignment, Σ_a	`alignPatch` in `tracking.cpp`
noise of each match	`matchNoise` in `main.cpp`, `EKF::setNoises`
adding, removing features	`Mapper::addFeatures`, `Mapper::removeUnreliable`
descriptors	`describe`, `Mapper::recordAppearance`
keyframes	`Keyframes` in `relocalization.h`
global matching	`matchGlobally` in `relocalization.h`
input, output	`FrameSource`, `StateStream`
window	`Display`, `draw` in `display.h`

References

[1] S. Baker and I. Matthews. Lucas-Kanade 20 years on: a unifying framework. *International Journal of Computer Vision*, 56(3):221–255, 2004.

[2] J. Civera, A. J. Davison and J. M. M. Montiel. Inverse depth parametrization for monocular SLAM. *IEEE Transactions on Robotics*, 24(5):932–945, 2008.

[3] J. Civera, O. G. Grasa, A. J. Davison and J. M. M. Montiel. 1-point RANSAC for extended Kalman filtering: application to real-time structure from motion and visual odometry. *Journal of Field Robotics*, 27(5):609–631, 2010.

[4] A. J. Davison, I. D. Reid, N. D. Molton and O. Stasse. MonoSLAM: real-time single camera SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1052–1067, 2007.

[5] G. Klein and D. Murray. Parallel tracking and mapping for small AR workspaces. In *Proceedings of the IEEE and ACM International Symposium on Mixed and Augmented Reality*, pages 225–234, 2007.

[6] G. Klein and D. Murray. Improving the agility of keyframe-based SLAM. In *Proceedings of the European Conference on Computer Vision*, pages 802–815, 2008.

[7] E. J. Lefferts, F. L. Markley and M. D. Shuster. Kalman filtering for spacecraft attitude estimation. *Journal of Guidance, Control, and Dynamics*, 5(5):417–429, 1982.

[8] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the 7th International Joint Conference on Artificial Intelligence*, pages 674–679, 1981.

[9] E. Rublee, V. Rabaud, K. Konolige and G. Bradski. ORB: an efficient alternative to SIFT or SURF. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2564–2571, 2011.

[10] J. Shi and C. Tomasi. Good features to track. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 593–600, 1994.

[11] H. Strasdat, J. M. M. Montiel and A. J. Davison. Visual SLAM: why filter? *Image and Vision Computing*, 30(2):65–77, 2012.

[12] B. Williams, G. Klein and I. Reid. Automatic relocalization and loop closing for real-time monocular SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(9):1699–1712, 2011.